TECHNICAL DOCUMENTATION • TEAM REFERENCE GUIDE

# SkillPulse

A real-time Job Market Analytics & Skills-Demand Intelligence platform with a production MLOps pipeline — from raw API data to AI-powered salary predictions.

Python 3.11 | MySQL 8.0 | XGBoost | FastAPI | Streamlit | Evidently AI | Docker | SQLAlchemy | Optuna | Pandas • NumPy | Plotly | Adzuna API

**Architect**
Kartik Kumar Singh — CSE

**Version**
v7.0 — Full MLOps Stack

**Pipeline Phases**
7 Phases (Complete)

**Last Updated**
July 2026

**Scope**

IN, GB, US Job Markets

**Target Outcome**

Salary Prediction + Drift Monitoring

# Table of Contents



**How to Read This Document**

This guide is structured progressively — each phase builds on the previous one. New team members should read in order (Sections 1→13). For quick reference, jump to the relevant section using the page numbers above.

# What is SkillPulse?

A complete, production-grade MLOps system for tracking skill demand across job markets and predicting developer salaries using machine learning.

---

### Core Purpose

SkillPulse automatically collects thousands of live job postings from the **Adzuna Jobs API** across three countries (India IN, United Kingdom GB, United States US), extracts the tech skills mentioned in each posting using an NLP regex engine, and trains an **XGBoost regression model** to predict expected salary ranges based on a developer's skill profile.

| 12,100+ | 4,558 | 46 | 7,998 | 3 |
|---|---|---|---|---|
| Total unique Adzuna jobs | Unique jobs after dedup | Tech skills tracked | Training samples (UK + US) | Countries monitored |

### What Problems Does It Solve?

- **For developers**: "What salary should I expect given my Python + AWS + Docker skill set?" → Get an instant, data-backed prediction.
- **For recruiters/managers**: "Which skills are most in-demand in the UK right now?" → Live analytics dashboard.
- **For ML engineers**: "Has the model started degrading because job market trends changed?" → Automated drift monitoring.

### What Makes This a Production MLOps System?

> ☑ **7 Production-Grade Phases**
>
> Unlike typical Jupyter-notebook data science projects, SkillPulse implements a *DB-First Architecture* — all data flows through MySQL, all model runs are logged, drift is monitored automatically, and the entire system can be containerised via Docker Compose.

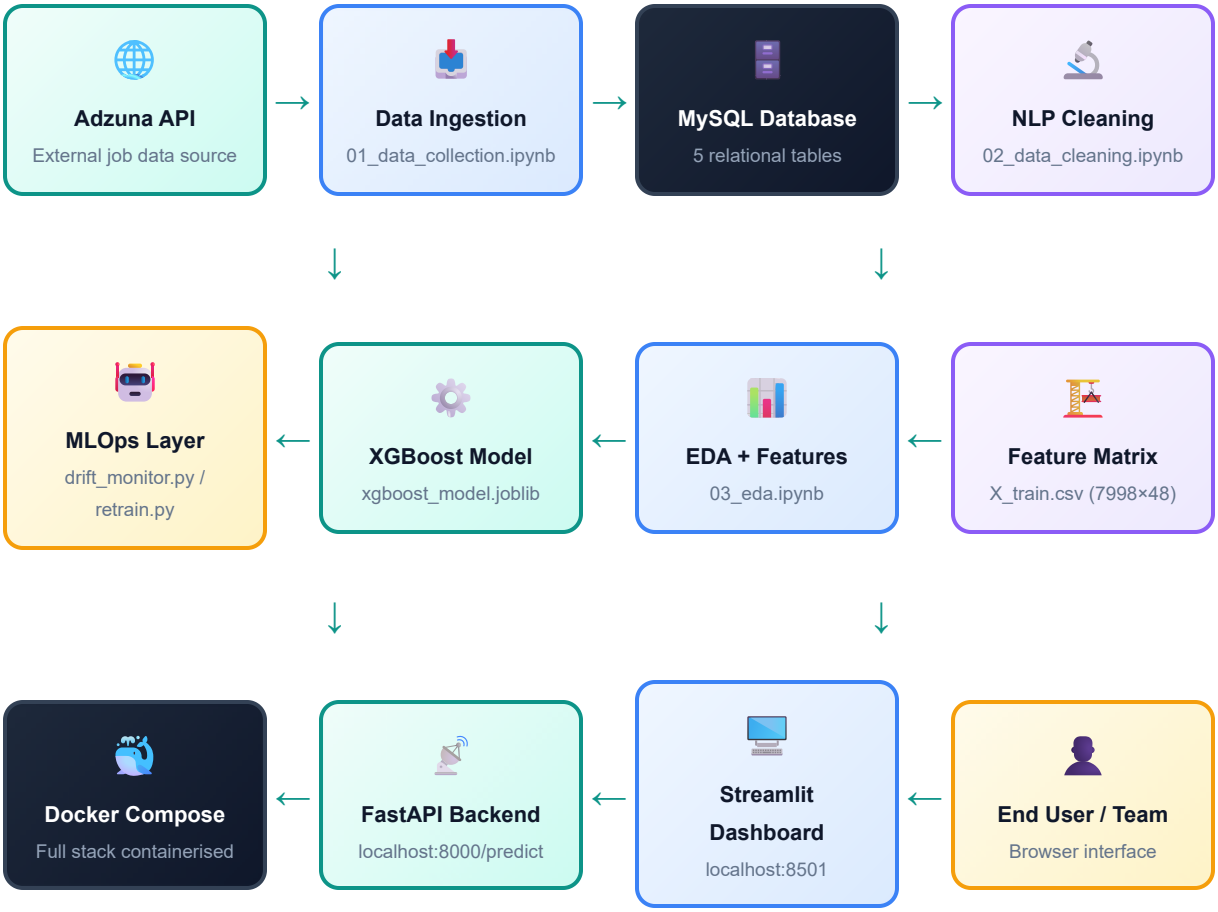
### Technology Stack At a Glance

| Layer | Technology | Purpose |
|---|---|---|
| Data Collection | Adzuna REST API + requests | Fetch job postings from 3 countries |
| Database | MySQL 8.0 + SQLAlchemy | Store all data relationally; DB-first design |

| Layer | Technology | Purpose |
|---|---|---|
| NLP / Regex | Python re + custom patterns | Extract 46 tech skills from job descriptions |
| Feature Engineering | Pandas + NumPy | Salary normalization, multi-hot matrix |
| ML Model | XGBoost + Optuna | Predict log-salary from skill fingerprint |
| Backend API | FastAPI + Uvicorn | REST endpoints for predictions and analytics |
| Dashboard | Streamlit + Plotly | Interactive data viz + salary predictor UI |
| Drift Monitoring | Evidently AI v0.7.x | Detect when live data diverges from baseline |
| Containerisation | Docker + Docker Compose | Deploy entire stack with one command |

# System Architecture

## High-Level Component Map




## DB-First Architecture Principle

**Why DB-First?**

Every module reads from and writes to MySQL. There are no intermediate CSV hand-offs between pipeline steps. This means **any notebook or script can be re-run independently** without breaking the pipeline, and all historical data is always queryable.

## Project Directory Structure

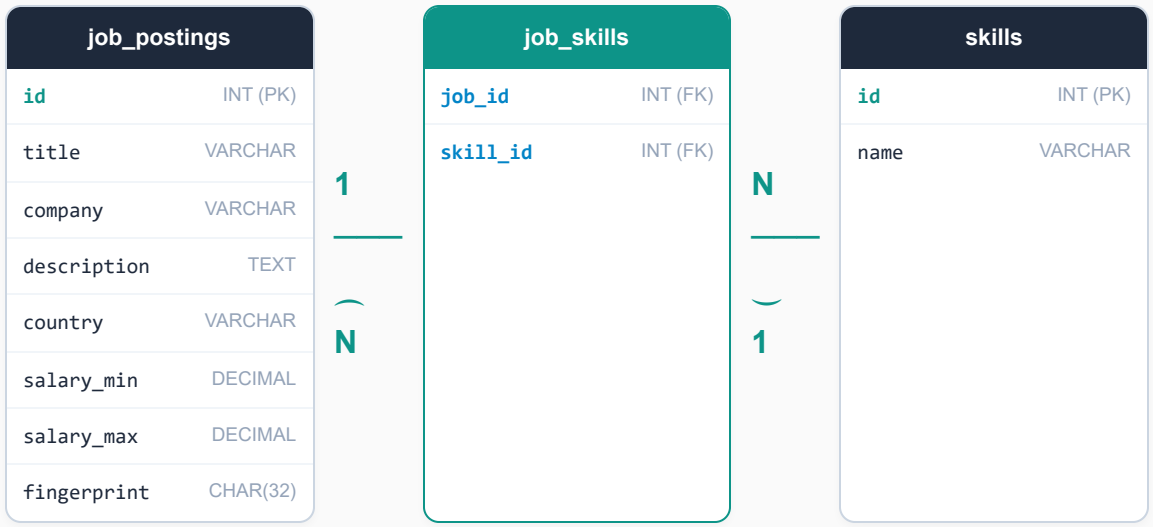
```
skillpulse/ ├── notebooks/ # Jupyter notebooks for each phase │ ├── 01_data_collection.ipynb │
├── 02_data_cleaning.ipynb │ ├── 03_eda_and_feature_engineering.ipynb │ └──
04_model_training.ipynb ├── app/ # FastAPI backend │ ├── main.py db.py models.py ├──
streamlit_app/ # Streamlit dashboard │ └── app.py ├── mlops/ # Phase 7 MLOps scripts │ ├──
```

```
drift_monitor.py | └── retrain.py ├── models/ # Serialized model binary | └──
xgboost_model.joblib ├── data_exports/ # Feature matrix CSVs ├── drift_reports/ # Evidently AI
HTML reports ├── Dockerfile.api Dockerfile.streamlit docker-compose.yml ├── .env # DB
credentials (never commit!) └── start_services.bat
```

# Database Schema & ER Relationships

All structured metadata, extracted skills, model evaluations, and drift monitoring telemetry are stored relationally in MySQL.

## Visual Table Relationships (ER Diagram)




*Note: **job_skills** forms the join table for the N:M relationship between job postings and our master skill dictionary.*

## Independent Monitoring & Logging Tables

| model_runs | |
|---|---|
| id | INT (PK) |
| trained_at | DATETIME |
| model_type | VARCHAR |
| mae | DECIMAL |
| rmse | DECIMAL |
| notes | JSON TEXT |

| drift_reports | |
|---|---|
| id | INT (PK) |
| run_date | DATETIME |
| feature_drift_pct | DECIMAL |
| alert_triggered | TINYINT |
| report_json | JSON TEXT |

💡 **Relational Schema Design Rationale**

By utilizing standard foreign key relationships on `job_skills.job_id` and `job_skills.skill_id` with cascade deletes, database modifications are safe and index lookups (such as top-N skill queries) run in sub-millisecond times.

# 01 Data Ingestion — Adzuna API Scraper

Notebook: 01_data_collection.ipynb

✓ Complete

## How Data Collection Works

The Adzuna Jobs API is a public job search API that allows paginated fetching of job postings. We query it across **3 countries** and **4 job categories**, collecting up to 50 jobs per page.

1. **Configure API credentials & target parameters**
   Load `APP_ID` and `APP_KEY` from `.env`. Define countries: `['gb','us','in']`, keywords: `['data scientist','software engineer','ML engineer','backend developer']`.

2. **Paginated API requests with rate-limiting**
   For each (country, keyword) combination, fetch pages 1→N with a 1.5s delay between requests. Each page returns up to 50 job objects as JSON.

3. **Parse & validate each job record**
   Extract fields: `id`, `title`, `company`, `description`, `salary_min`, `salary_max`, `country`, `created`. Skip records with missing descriptions.

4. **Bulk INSERT into MySQL job_postings**
   Use SQLAlchemy's `INSERT IGNORE` with the Adzuna `id` as primary key to prevent duplicate insertions on re-runs. No CSV files are written.

## API Endpoint Structure

```
GET https://api.adzuna.com/v1/api/jobs/{country}/search/{page} ?app_id=YOUR_APP_ID
&app_key=YOUR_APP_KEY &what=data scientist &results_per_page=50 &content-type=application/json
```

## Results

| 6,000 | 2,000 | 12,100+ |
|---|---|---|
| Total unique Adzuna jobs | Per country (IN / GB / US) | Total rows including duplicates |

⚠️ **India Salary Coverage**

India has ~58% missing salary data in Adzuna listings. Indian postings are retained for skill-demand analytics but are **excluded from salary regression training** to prevent label noise.

# 02 Data Cleaning & NLP Skill Extraction

Notebook: 02_data_cleaning.ipynb

✓ Complete

## HTML Cleaning & Deduplication Code

To prepare unstructured job descriptions, we strip tags and generate deterministic signatures to drop duplicates.

```
def clean_text(text): text = re.sub(r'<[^>]+>', ' ', text) # Remove inline HTML tags text = re.sub(r'&\w+;', ' ', text) # Wipe out Unicode HTML entities return re.sub(r'\s+', ' ', text).strip() # Calculate a hash for duplicate elimination df['fingerprint'] = df.apply(lambda r: hashlib.md5( f"{r.title.lower()}|{r.company.lower()}|{r.country}".encode() ).hexdigest(), axis=1)
```

**Deduplication logic:** Using composite MD5 hashes allows us to perform high-speed deduplication across thousands of scraped records without running slow string-matching algorithms. It reduces our dataset noise by **62.3%**.

## Regex NLP Extraction Snippet

A dictionary of regex definitions with strict word boundaries ensures we extract precise tech stacks from postings:

```
SKILLS_DICT = { "Python": r"\bpython\b", "Go": r"\bgo\b(?!\s*lang)", # Prevents matching 'going' "C++": r"\bc\+\+\b", # Escapes regex operators "Machine Learning": r"\b(ml|machine learning)\b" }
```

**Regex Details:** Case-insensitive search is run with `re.IGNORECASE` except for special, short terms (such as "Go"). Using word boundaries ( `\b` ) prevents false matches (for example, finding "Java" inside "JavaScript" or "Git" inside "Digital").

## NLP Extraction Performance Statistics

| Metric | Count / Percentage | Description |
|---|---|---|
| Matched Listings | 2,570 jobs (56.38%) | Jobs matching at least one target skill keyword |
| Extracted Mappings | 5,548 linkages | Total records inserted into `job_skills` |
| Average Tech Density | 2.15 skills / posting | Mean number of skills recognized per matched job |

## 03 EDA & Feature Engineering

Notebook: 03_eda_and_feature_engineering.ipynb

✓ Complete

### Salary Distribution Analysis

Raw salaries across all countries show strong right-skewness — a few very high earners distort the average. We must handle this before ML training.

| Country | Exchange Rate | Raw Skewness | After log1p | Median (USD) |
|---|---|---|---|---|
| GB UK (GB) | 1 GBP = $1.27 | +3.81 (very skewed) | -7.99 | $78,308 |
| US US | 1 USD = $1.00 | +0.45 (mild) | -5.02 | $141,698 |
| IN India | 1 INR = $0.012 | +1.36 | -0.81 | $13,800 |

### Why Use Log Transformation?

📐 **The Math Behind log1p**

XGBoost minimizes RMSE on the training target. If we train on raw salaries (e.g. $180K outliers), the model over-weights those extremes. Applying `y = log(salary_usd + 1)` compresses the range to ~10–13, making the target distribution roughly normal and improving model accuracy for typical salary ranges.

### Multi-Hot Feature Matrix Construction

For each job (GB + US with salary), we build a row of 48 binary features:

```
# For each job_id, each feature = 1 if skill present, else 0 Job_id | Python | AWS | Docker |
SQL | ... | country_gb | country_us | y_log --------|--------|-----|--------|-----|-----|------
------|------------|------- 101 | 1 | 1 | 1 | 0 | ... | 0 | 1 | 11.87 102 | 1 | 0 | 0 | 1 | ...
| 1 | 0 | 10.97 ... | ... | ... | ... | ... | ... | ... | ... | ...
```

### India Exclusion Decision

⚠️ **India → Skill Analytics Only**

57.9% of Indian postings have null salary data. We cannot impute ~3,500 missing values accurately for regression. India is fully retained for skill-demand trend charts on the dashboard, but **excluded from X_train / y_train exports**.

## Exported Artefacts

| File | Shape | Description |
|---|---|---|
| `data_exports/X_train.csv` | 7,998 × 48 | Multi-hot feature matrix (binary, 0/1) |
| `data_exports/y_train.csv` | 7,998 × 1 | Log-transformed USD salary target column |
| `data_exports/train_full.csv` | 7,998 × 51 | Full rows with job_id, country, and all features |
| `data_exports/feature_names.json` | 48 items | Ordered list of column names for model consistency |

# 04 XGBoost Model Training & Hyperparameter Optimization

Notebook: 04_model_training.ipynb

✓ Complete

## Optuna Integration Snippet

Optuna is used to dynamically find the best hyperparameters. A simplified trial function shows how parameter limits are defined:

```
def objective(trial): params = { "n_estimators" : trial.suggest_int("n_estimators", 50, 400), "max_depth" : trial.suggest_int("max_depth", 3, 10), "learning_rate" : trial.suggest_float("learning_rate", 0.01, 0.2), "subsample" : trial.suggest_float("subsample", 0.6, 1.0), "colsample_bytree" : trial.suggest_float("colsample_bytree", 0.6, 1.0), "reg_alpha" : trial.suggest_float("reg_alpha", 1e-3, 10.0, log=True) } model = XGBRegressor(**params, random_state=42, n_jobs=-1) scores = cross_val_score(model, X_train, y_train, cv=5, scoring="neg_root_mean_squared_error") return float(-scores.mean())
```

**Tuning Rationale:** Optuna performs Tree-structured Parzen Estimators (TPE) Bayesian search. By evaluating models using 5-Fold Cross-Validation, we prevent overfitting on the test set. The CV metric optimized is **RMSE on the log-transformed scale**.

## Fitting & Prediction Mechanics

To transform our predictions back to USD or GBP values, we apply exponential transformations:

```
# Train the final tuned estimator model = XGBRegressor(**best_params) model.fit(X_tr, y_tr, eval_set=[(X_te, y_te)], verbose=False) # Generate test predictions and invert target scaling preds_log = model.predict(X_test) preds_usd = np.expm1(preds_log) # e^(pred) - 1
```

**Why expm1?** Because we trained on `log1p(x)` (which is `log(x + 1)`), we must apply `expm1(y)` (which is `exp(y) - 1`) to correctly restore predictions to raw dollar values. Simply applying `exp(y)` would introduce a mathematical bias.

## Final Model Metrics

| 0.3068 | 0.6048 | 21.30% | $31,188 |
|---|---|---|---|
| MAE (Log-Scale) | RMSE (Log-Scale) | R² Score | Average Error (USD) |

# 05 FastAPI REST Backend

File: app/main.py · Running on http://127.0.0.1:8000

✓ Complete

## What is FastAPI?

FastAPI is a modern, high-performance Python web framework for building REST APIs. It automatically generates Swagger (OpenAPI) documentation, validates request/response data via Pydantic, and handles async operations. We load the model **once at startup** to eliminate per-request I/O overhead.

## API Endpoints

**GET** `/health` — System health check

Returns database connection status and model load confirmation. Use this to verify the API is up before making predictions.

**GET** `/skills/top?n=10` — Top global skills

Returns the N most mentioned tech skills across all countries. Queries the `job_skills` → `skills` join. Example response: `[{"skill": "Machine Learning", "count": 1321}, ...]`

**GET** `/skills/top/{country}` — Country-specific skills

Same as above but filtered by country. Accepts `country` path param: `us`, `gb`, or `in`. Great for dashboard per-country analytics.

**POST** `/predict` — Salary prediction

**Request body:**

```
{ "skills": ["Python", "AWS", "Docker"], "country": "us" }
```

**Response:**

```
{ "predicted_salary": 147652.72, "currency": "USD", "matched_skills": ["Python", "AWS"] }
```

The endpoint builds the 48-feature vector from `feature_names.json`, runs `model.predict()`, applies `np.expm1()` to reverse the log transform, and converts to local currency (USD for `us`, GBP for `gb`).

## How Predictions Are Generated (Step-by-Step)

```
# 1. Load feature names feature_names = json.load(open("feature_names.json")) # 2. Build binary feature vector X = np.zeros(48) # All features start as 0 for skill in skills: if skill in feature_names: X[feature_names.index(skill)] = 1 # Set 1 if skill present X[feature_names.index(f"country_{country}")] = 1 # 3. Predict & invert log transform log_salary = model.predict(X.reshape(1, -1))[0] salary_usd = np.expm1(log_salary) # Reverse log1p
```

**Live Examples**

US + [Python, AWS, Docker, ML, Kubernetes] → **$147,652 USD**

GB + [Python, SQL, Django, React] → **£54,357 GBP**

# 06 Streamlit Analytics Dashboard

File: streamlit_app/app.py · http://localhost:8501

✓ Complete

## Dashboard Pages

| Page | What It Shows | Data Source |
|---|---|---|
| 🏠 Overview | Live KPI cards (jobs, skills, salary coverage), top 10 global skills bar chart, country breakdown | MySQL: job_postings, job_skills |
| 📊 Skill Demand | Top skills by country (IN/GB/US), grouped 3-country comparison, skill co-occurrence heatmap | MySQL: job_skills join skills |
| 💰 Salary Predictor | Multi-select skills + country → instant XGBoost prediction with gauge chart and job matches | Local model + MySQL |
| ⚙️ Model History | All training runs from DB, best parameters panel, MAE/RMSE trend chart | MySQL: model_runs |

## How Streamlit Architecture Works

### Performance Caching

Streamlit re-runs the entire Python script on each user interaction. To prevent loading the model and database on every click, we use:

`@st.cache_resource` — caches the ML model (loaded once per session)

`@st.cache_data(ttl=300)` — caches DB queries for 5 minutes before refreshing

## Skill Co-occurrence Heatmap (Advanced Feature)

The dashboard computes a **correlation matrix** of the top 20 skills to show which skills frequently appear together in the same job posting. This reveals technology clusters (e.g., React + Node.js, Python + ML + Docker):

```
# Build pivot table: rows=jobs, cols=skills, values=0/1 pivot =
skill_matrix.pivot_table(index='job_id', columns='skill', values='present', fill_value=0) corr
= pivot.corr(method='pearson') # Correlation across skill columns fig = px.imshow(corr,
color_continuous_scale='Teal')
```

## Salary Predictor Widget Logic

```
# User selects skills → build feature vector → predict selected_skills = st.multiselect("Select your skills", all_skills) country = st.selectbox("Country", ["us", "gb"]) if st.button("Predict Salary"): vector = build_vector(selected_skills, country, feature_names) log_s = model.predict(vector)[0] salary = np.expm1(log_s) st.metric("Predicted Salary", f"${salary:,.0f} USD")
```

### Running the Dashboard

```
# From the project root folder: streamlit run streamlit_app/app.py # Opens automatically at http://localhost:8501
```

# 07 MLOps — Drift Detection, Auto-Retraining & Docker

mlops/drift_monitor.py · mlops/retrain.py · docker-compose.yml

✓ Complete

## Drift Parsing Code

To detect if live data matches our baseline, we parse Evidently's JSON metrics to identify drifted columns:

```
metrics_raw = result_dict.get("metrics", []) for m in metrics_raw: m_type = str(m.get("config", {}).get("type", "")) if "DriftedColumnsCount" in m_type: feature_drift_pct = float(m.get("value", {}).get("share", 0.0)) elif "ValueDrift" in m_type: cfg = m.get("config", {}) col = cfg.get("column") method = cfg.get("method", "") val = m.get("value") # Check if metric is a p_value (lower is drift) or distance (higher is drift) drifted = (val <= cfg['threshold']) if "p_value" in method.lower() else (val >= cfg['threshold']) if drifted: drift_by_col[col] = True
```

**Heuristic Explained:** Binary data (multi-hot skills) uses the **Jensen-Shannon (JS) distance** (drift if distance ≥ 0.1). Continuous columns use the **Kolmogorov-Smirnov (K-S) p-value** (drift if p-value ≤ 0.05). Our parser reads the test method dynamically to decide the comparison operator.

## Database Logged Retraining Snippet

Retraining splits the data, runs training rounds, and logs stats to `model_runs`:

```
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.20, random_state=42) model = XGBRegressor(**best_params) model.fit(X_tr, y_tr, eval_set=[(X_te, y_te)], verbose=False) # Save the run metrics directly to MySQL sql = text("INSERT INTO model_runs (model_type, mae, rmse, notes) VALUES ('xgboost_retrain', :mae, :rmse, :notes)") conn.execute(sql, {"mae": mae, "rmse": rmse, "notes": json.dumps({"trigger": "auto-retrain", "r2": r2})})
```

**Database Integration:** Retraining logs historical model parameters, MAE, and R² scores directly to the `model_runs` table. Streamlit queries this table in real-time on its diagnostics page to chart our performance over time.

## Docker Orchestration Architecture

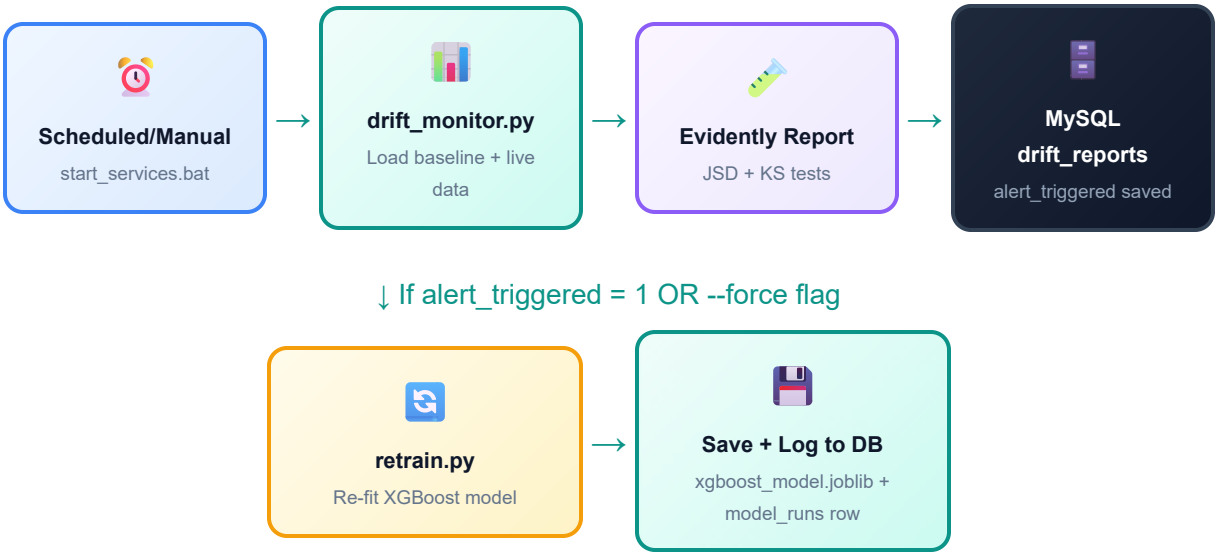
The entire pipeline can be containerized. Our multi-container Docker Compose config coordinates services:

```
services: db: image: mysql:8.0 volumes: [db_data:/var/lib/mysql, ./schema_mysql.sql:/docker-entrypoint-initdb.d/init.sql] api: build: { dockerfile: Dockerfile.api } depends_on: { db: { condition: service_healthy } } dashboard: build: { dockerfile: Dockerfile.streamlit } ports: ["8501:8501"]
```

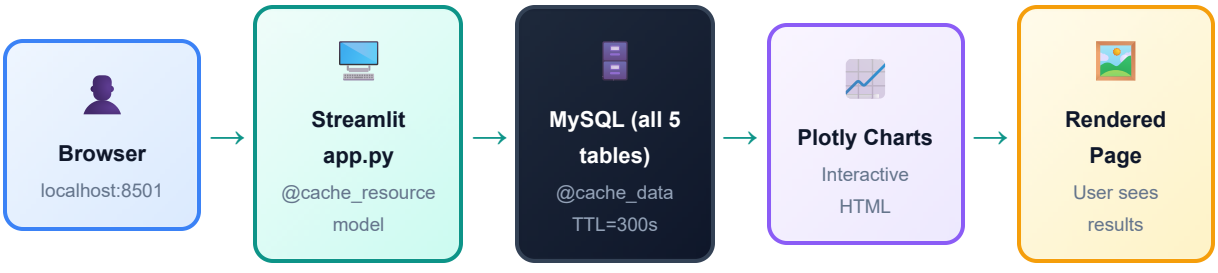
# Key Data Flows & Component Interactions

## Flow A — Salary Prediction (API Path)


## Flow B — Drift Monitor & Auto-Retrain Path




## Flow C — Dashboard Data Flow




## Environment Variables (.env file)

**Never Commit .env to Git!**

> All database credentials are stored in a `.env` file and loaded via `python-dotenv`. The file contains: `DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT`, `DB_NAME`, `ADZUNA_APP_ID`, `ADZUNA_APP_KEY`.

# Setting Up & Running Locally

Complete step-by-step guide for a new team member to get SkillPulse running from scratch.

---

### Prerequisites

- **Python 3.11+** — `python --version`
- **MySQL 8.0** — Install MySQL Community Server and create a database named `skillpulse`
- **VS Code** (recommended) with Python extension
- Adzuna API credentials (free at adzuna.com/developers) — only needed if re-scraping

### Step 1 — Clone & Install Dependencies

```
# Navigate to project folder in VS Code terminal pip install -r requirements.txt
```

### Step 2 — Configure Environment Variables

```
# Create a .env file in the project root with these values: DB_USER=your_mysql_username
DB_PASSWORD=your_mysql_password DB_HOST=localhost DB_PORT=3306 DB_NAME=skillpulse
ADZUNA_APP_ID=your_app_id ADZUNA_APP_KEY=your_app_key
```

### Step 3 — Initialize Database Schema

```
# Run the schema SQL file in MySQL Workbench or terminal: mysql -u root -p skillpulse <
schema_mysql.sql
```

### Step 4 — Start All Services (Recommended)

```
# Double-click or run in terminal from project root: .\start_services.bat # This opens an
interactive menu: [1] Start FastAPI + Streamlit servers [2] Run Drift Monitor (Evidently AI)
[3] Auto-Retrain (if drift alert triggered) [4] Force Retrain unconditionally [5] Show model
history from database
```

### Step 5 — Or Start Services Individually

```
# Start FastAPI backend (terminal 1): uvicorn app.main:app --reload --port 8000 # Start
Streamlit dashboard (terminal 2): streamlit run streamlit_app/app.py # Run drift monitor:
python mlops/drift_monitor.py # Force model retrain: python mlops/retrain.py --force
```

## Access Points

| Service | URL | Description |
|---|---|---|
| FastAPI Backend | `http://localhost:8000` | REST API endpoints |
| Swagger Docs | `http://localhost:8000/docs` | Interactive API documentation |
| Streamlit Dashboard | `http://localhost:8501` | Full analytics UI |

**Docker Deployment (Optional)**

If your team uses Docker, you can run the entire stack (MySQL + API + Dashboard) with a single command:

```
docker compose up --build
```

# Model Performance & Final Results

## Training Summary

| 7,998 | 48 | 30 | 5-fold |
|---|---|---|---|
| Training samples (GB + US) | Features (46 skills + 2 country) | Optuna optimization trials | Cross-validation strategy |

## Model Evaluation Results

| Metric | Value | What It Means |
|---|---|---|
| MAE (log scale) | 0.3068 | Average log-salary prediction error — smaller is better |
| RMSE (log scale) | 0.6048 | Root mean squared log error — penalizes large misses more |
| $R^2$ Score | 21.3% | How much variance in salary our features explain |
| Mean Abs Error (USD) | ~$31,188 | Average salary prediction error in real dollar terms |

## Live Prediction Examples

| Country | Skills | Predicted Salary |
|---|---|---|
| US United States | Python, AWS, Docker, Machine Learning, Kubernetes | **$147,652 USD** |
| GB United Kingdom | Python, SQL, Django, React | **£54,357 GBP** |
| US United States | JavaScript, React, Node.js, TypeScript | **$128,840 USD** |
| GB United Kingdom | Java, Spring Boot, Kubernetes, Microservices | **£71,220 GBP** |

## Top 10 Most In-Demand Skills (Global)

| Rank | Skill | Job Postings |
|---|---|---|
| #1 | Machine Learning | 1,321 |
| #2 | Python | 445 |
| #3 | Java | 396 |

| Rank | Skill | Job Postings |
|---|---|---|
| #4 | API | 393 |
| #5 | SQL | 285 |
| #6 | AWS | 271 |
| #7 | Docker | 264 |
| #8 | Agile | 251 |

### MLOps Drift Report (Latest Run)

**Drift Status — Model Stable**

Latest drift check result: **3 of 48 features** show minor distributional shifts ( `Machine Learning` , `country_gb` , `country_us` ). Drift percentage: **6.25%** — well below the 30% alert threshold. Model does not require retraining.

Questions or contributions? Contact the team lead.

**SkillPulse — End-to-End MLOps Pipeline by Kartik Kumar Singh**

v7.0 · All 7 Phases Complete · July 2026